

National University of Computer and Emerging  
Sciences



**Database Systems Lab**

**Project Deliverable**

**Team Composition:**

| Member Name   | Member Roll # |
|---------------|---------------|
| Muhammad Umer | 23L-3085      |
| Talha Farooq  | 23L-3074      |
| Usama Iqbal   | 22L-7970      |

## **Project Details:**

- **Project Name:**  
**VroomVault**

- **Project Idea:**

**VroomVault** is a smart Parking Management System designed to efficiently manage parking spaces in an area with a limited number of slots. The system will track available and occupied slots, assist users in finding and reserving spaces, and maintain parking data through a real-time database. It will also handle fee collection as well as fine generation for late departures, ensuring smooth and organized parking operations.

- **Project About:**

VroomVault is a car parking management system that helps organize and monitor parking spaces for vehicles. It provides real-time updates on available slots, allows users to reserve spaces easily, and automates fee and fine management. The project aims to make parking more efficient, reliable, and convenient for both users and administrators.

# Queries along Functionalities:

## 1. InsertAdminDetails

### Description:

Inserts a new Admin's CNIC, phone number, and address into the Admins\_Details table and username and password into the Admin\_Logins table.

```
Create procedure InsertAdminDetails
    @AdminCNIC VARCHAR(20),
    @AdminPhoneNo VARCHAR(15),
    @AdminAddress TEXT,
    @AdminName VARCHAR(100),
    @Password VARCHAR(10)
as
Begin
    Insert into dbo.Admins_Details(AdminCNIC, AdminPhoneNo,
AdminAddress)
    values (@AdminCNIC, @AdminPhoneNo, @AdminAddress);

    Insert into Admin_Logins (AdminCNIC, AdminName, Password)
    values (@AdminCNIC, @AdminName, @Password);

    Select 'Admin account created successfully' AS Result;
End;
```

---

## 2. UpdateAdminPassword

### Description:

Updates the admin's password if the old password matches and the new password is different from the old one.

```
Create procedure UpdateAdminPassword
    @AdminCNIC varchar(20),
    @OldPassword varchar(10),
    @NewPassword varchar(10)
AS
Begin
    if exists (
        Select 1 from Admin_Logins where AdminCNIC = @AdminCNIC AND
Password = @OldPassword and @OldPassword <> @NewPassword
    )
        Begin
            Update Admin_Logins
            set Password = @NewPassword
            where AdminCNIC = @AdminCNIC;

            select 'Password updated successfully' AS Result;
        end
    else
        begin
            select 'Invalid credentials' AS Result;
        end
    end
End;
```

---

### 3. findAdmin

**Description:**

Validates admin login by checking CNIC and password. If valid, fetches admin details (name, CNIC, phone number, and address).

```
Create procedure findAdmin
    @AdminCNIC varchar(20),
    @Password varchar(10)
as
begin
    If exists(Select 1 from Admin_Logins where AdminCNIC = @AdminCNIC
and Password = @Password)
        Begin
            Select dbo.Admin_Logins.AdminName, dbo.Admin_Logins.AdminCNIC,
dbo.Admins_Details.AdminPhoneNo, dbo.Admins_Details.AdminAddress
            from Admin_Logins
            join Admins_Details  ON dbo.Admin_Logins.AdminCNIC =
dbo.Admins_Details.AdminCNIC
            where dbo.Admin_Logins.AdminCNIC = @AdminCNIC;

            return 1;
        end
    else
        begin
            Select 'Admin Not found within these Details' AS Result;
            return 0;
        end
    end
End;
```

---

#### 4. Stored Procedure: InsertUserDetails

**Description:** This procedure inserts a new user into the Users table after checking for duplicate CNIC or phone number.

```
go
```

```
Create Procedure InsertUserDetails
```

```
    @UserName varchar(100),
```

```
    @UserCNIC varchar(20),
```

```
    @UserPhoneNo varchar(15)
```

```
AS
```

```
Begin    -- checkfor duplicate
```

```
    IF exists (SELECT 1 FROM Users WHERE UserCNIC = @UserCNIC OR  
UserPhoneNo = @UserPhoneNo)
```

```
    begin
```

```
        select 'Duplicate CNIC or Phone Number detected' AS  
Result;
```

```
    end
```

```
    else
```

```
    Begin
```

```
        Insert into Users (UserName, UserCNIC, UserPhoneNo) values  
(@UserName, @UserCNIC, @UserPhoneNo);
```

```
SELECT 'User added successfully' AS Result;
```

```
END
```

END;

go

---

## 5. Stored Procedure: InsertVehicleDetails

**Description:** This procedure inserts a vehicle's details into the Vehicles table after validating the category.

```
Create Procedure InsertVehicleDetails
```

```
    @LicenseNo varchar(20),
```

```
    @Category varchar(10)
```

```
AS
```

```
Begin    -- inserting vehicle
```

```
    if @Category NOT IN ('Sedan', 'SUV', 'Compact')
```

```
    begin
```

```
        Select 'Invalid category. Must be a Sedan, SUV, or  
Compact.' AS Result;
```

```
        return;
```

```
    end
```

```
    Insert into Vehicles (LicenseNo, Category)
```

```
    values (@LicenseNo, @Category);
```

```
    Select 'Vehicle added successfully' AS Result;
```

```
end;
```

```
Go
```

---



## 6. Stored Procedure: InsertReservationDetails

**Description:** This procedure inserts reservation details, checks if arrival and departure times are valid, and links to the corresponding time interval record.

```
Create procedure InsertReservationDetails
```

```
    @P_LicenseNo varchar(20) ,
```

```
    @P_SlotNumber int ,
```

```
    @P_UserCNIC varchar(20) ,
```

```
    @P_ArrivalTime datetime ,
```

```
    @P_DepartureTime datetime
```

```
As
```

```
Begin -- inserting the current reservation
```

```
    If ( @P_LicenseNo is Null or @P_SlotNumber is null or @P_UserCNIC is  
        NULL or @P_ArrivalTime is null or @P_DepartureTime is NULL )
```

```
Begin
```

```
    Print 'Please enter valid data to reserve a slot!!!'
```

```
    return;
```

```
End
```

```
IF @P_DepartureTime <= @P_ArrivalTime
```

```
BEGIN
```

```
    SELECT 'Departure time must be later than arrival time' AS Result;
```

```
    RETURN;
```

END

```
Insert into dbo.Reservation_Details (LicenseNo , SlotNumber , UserCNIC
, PaymentStatus) values ( @P_LicenseNo , @P_SlotNumber , @P_UserCNIC ,
'Unpaid' );
```

```
Declare @P_RID int = 0;
```

```
Select @P_RID = dbo.Reservation_Details.ReservationId from
dbo.Reservation_Details where dbo.Reservation_Details.LicenseNo =
@P_LicenseNo
```

```
and dbo.Reservation_Details.SlotNumber = @P_SlotNumber and
dbo.Reservation_Details.PaymentStatus = 'Unpaid' and
dbo.Reservation_Details.UserCNIC
```

```
= @P_UserCNIC
```

```
Insert into dbo.Reservation_TimeInterval_Record values ( @P_RID ,
@P_ArrivalTime , @P_DepartureTime );
```

```
Select 'Reservation added successfully' AS Result;
```

```
End;
```

---

## 7. Stored Procedure: VehicleCheck

**Description:** This procedure checks if there are active reservations for a given vehicle based on its license number.

```
create procedure VehicleCheck    -- check if reservation with that  
vehicle exists or not
```

```
    @LicenseNumber varchar(20)
```

```
as
```

```
begin
```

```
    IF exists (
```

```
        Select 1 from dbo.Vehicles
```

```
        where dbo.Vehicles.LicenseNo = @LicenseNumber
```

```
    )
```

```
    BEGIN
```

```
        SELECT 'active reservations found for this vehicle' AS  
Result;
```

```
        Print 'Active Reservation Found'
```

```
        RETURN 1;
```

```
    END
```

```
    ELSE
```

```
    BEGIN
```

```
        SELECT 'No active reservations found for this vehicle' AS  
Result;
```

```
        RETURN 0;
```

END

END;

Drop procedure VehicleCheck

EXECUTE VehicleCheck 'LEE\_677';

select \* from Reservation\_Details

select \* from dbo.Vehicles

---

## 8. Stored Procedure: BookParkingSlot

- **Description:** This procedure checks if a parking slot is available for booking and ensures the slot number is valid.

```
go
```

```
CREATE PROCEDURE BookParkingSlot  -- book the parking slot
    @Category VARCHAR(20),
    @SlotNumber INT
AS
BEGIN
    -- Check if slot is available
    IF NOT EXISTS (
        SELECT 1 FROM ParkingSlots
        WHERE SlotNumber = @SlotNumber AND [Status] = 'Available'
        -- check if that slot is available or not
    )
    BEGIN
        SELECT 'Selected slot is not available' AS Result;
        RETURN 0;
    END
END
```

```
Return 1;
```

```
END;
```

```
EXEC BookParkingSlot
```

```
    @Category = 'SUV',
```

```
    @SlotNumber = 23
```

```
Go
```

---

## 9. Stored Procedure: CalculateFine

**Description:** This procedure calculates the fine for a vehicle based on how late it is compared to the promised departure time.

```
Create procedure CalculateFine
```

```
    @LicensoNo varchar(20) ,
```

```
    @ActualTime datetime
```

```
As
```

```
Begin
```

```
    Declare @ActualArrivalTime datetime ;
```

```
    Declare @DepartArrivalTime datetime ;
```

```
    Declare @P_ReservationId int ;
```

```
    Declare @FineAmount Decimal(10,2) = 0; -- take current time and  
    check how much time user is late from promised departure time
```

```
    Select @P_ReservationId = dbo.Reservation_Details.ReservationId  
    from dbo.Reservation_Details where  
    dbo.Reservation_Details.LicenseNo = @LicensoNo;
```

```
    Select @ActualArrivalTime =  
    dbo.Reservation_TimeInterval_Record.ArrivalTime ,  
    @DepartArrivalTime =  
    dbo.Reservation_TimeInterval_Record.DepartureTime
```

```
    from dbo.Reservation_TimeInterval_Record where  
    dbo.Reservation_TimeInterval_Record.ReservationId  
    =@P_ReservationId ;
```

```
If (@ActualTime between @ActualArrivalTime AND
@DepartArrivalTime)

    Begin

        SET @FineAmount = 0;

    End

Else If (@ActualTime > @DepartArrivalTime)

    Begin

        Declare @MinutesLate Int;

        Set @MinutesLate = DATEDIFF(MINUTE, @DepartArrivalTime,
@ActualTime);

        Set @FineAmount = @MinutesLate * 0.5;

    End

    Update dbo.Reservations_Charges set
    dbo.Reservations_Charges.Fine = @FineAmount where
    dbo.Reservations_Charges.ReservationId = @P_ReservationId;

END

Drop procedure CalculateFine
```



```
Declare @timenow datetime = getdate();
```

```
Exec CalculateFine 'LEE_678',@timenow
```

---

## 10. Stored Procedure: CalculateParkingFee

**Description:** This procedure calculates the parking fee for a reservation based on the arrival and departure times. The fee is calculated per minute, at a rate of 0.5 currency units per minute.

```
go
```

```
Create Procedure CalculateParkingFee
```

```
    @ArrivalDate Datetime,
```

```
    @DepartureDate Datetime,
```

```
    @ReservationID int
```

```
As Begin
```

```
    Declare @ParkingFeeRate Decimal(10,2) = 0.5;
```

```
    Declare @TotalMinutes Int;
```

```
    Declare @ParkingFee Decimal(10,2);
```

```
    If @ReservationId is not null
```

```
    Begin    -- calculate parking fee as per minute = 0.5 rupess
```

```
        Set @TotalMinutes = DATEDIFF(MINUTE, @ArrivalDate,  
@DepartureDate);
```

```
        Set @ParkingFee = @TotalMinutes * @ParkingFeeRate;

        Insert into dbo.Reservations_Charges (ReservationId,
ParkingFee, Fine)

            values (@ReservationId, @ParkingFee, NULL);    --
inserting null at fine since cant calculate it now

    end

else

begin

    Print 'Reservation not found for given arrival and
departure times.';

    end

End;
```

---

## 11. Trigger: trg\_CalculateParkingFee

**Description:** This trigger is executed after an insert on `Reservation_TimeInterval_Record`. It calls the `CalculateParkingFee` procedure to calculate the parking fee for the new reservation.

```
go
```

```
Create trigger trg_CalculateParkingFee
```

```
ON dbo.Reservation_TimeInterval_Record
```

```
After Insert
```

```
As
```

```
Begin
```

```
    Declare @ReservationId INT;
```

```
    Declare @ArrivalDate DATETIME;
```

```
    Declare @DepartureDate DATETIME;  -- this trigger automatically  
    initiate and call procedure that calculate the fine and insert at the
```

```
                                -- respective table
```

```
    Select @ReservationId =  
    dbo.Reservation_TimeInterval_Record.ReservationId ,@ArrivalDate =  
    dbo.Reservation_TimeInterval_Record.ArrivalTime, @DepartureDate =  
    dbo.Reservation_TimeInterval_Record.DepartureTime
```

```
    from inserted join dbo.Reservation_TimeInterval_Record On  
    dbo.Reservation_TimeInterval_Record.ReservationId =  
    inserted.ReservationId;
```

```
EXEC CalculateParkingFee  
@ArrivalDate,@DepartureDate,@ReservationId;  -- calculate parking fee  
procedure
```

```
END;
```

---

## 12. Stored Procedure: check\_slot\_status

- **Description:** This procedure checks the status of a parking slot (whether it's occupied or available). It also ensures the slot number does not exceed the total number of available slots (25).

go

```
Create procedure check_slot_status
```

```
    @DummySlotNumber int
```

```
As
```

```
Begin
```

```
    Declare @Full int = 0;
```

```
    Select @Full = count(*) from dbo.ParkingSlots where  
dbo.ParkingSlots.Status = 'Occupied' --counting slots that are  
already occupied
```

```
    IF(@Full >= 25 )
```

```
    Begin
```

```
        Print 'Sorry All the slots are Occupied.';
```

```
        return 0;    -- all slot reserved check
```

```
    End
```

```
IF (@DummySlotNumber > 25) -- greater than maximum slot check
```

```

Begin
    Print 'Slot number should not be greater than the total slots
25.';

    return 0;
end

declare @SlotStatus varchar(20);

Select @SlotStatus = dbo.ParkingSlots.Status
from dbo.ParkingSlots
where dbo.ParkingSlots.SlotNumber = @DummySlotNumber;

IF (@SlotStatus = 'Occupied')    -- check if slot already occupied
Begin
    Print 'Slot is already occupied.';

    return 0;
end

IF (@SlotStatus = 'Available') -- check if it not occupierd
Begin
    Print 'Slot is available.';

    return 1;
end

```

```
        end

        PRINT 'Invalid status or slot not found.';

        return 0;

END;

GO


Declare @preferedSlot int = 19;

Declare @ReturnValue int;

Exec @ReturnValue = check_slot_status @preferedSlot;

If( @ReturnValue = 1 )

Begin

    Print 'So Slot can be booked!!'

    Update dbo.ParkingSlots set dbo.ParkingSlots.Status = 'Occupied'
where dbo.ParkingSlots.SlotNumber = @preferedSlot;

End

Else

Begin

    Print 'So Slot cant be booked!!'

End

go
```

---



### 13. Function: getFees

**Description:** This function retrieves the parking fee for a given vehicle based on its license number.

go

```
Create function getFees(@LicNo varchar(20) )
```

```
returns int -- function that return the fees scalar
```

```
AS
```

```
Begin
```

```
Declare @P_ReservationId int = 0;
```

```
Select @P_ReservationId = dbo.Reservation_Details.ReservationId  
from dbo.Reservation_Details where  
dbo.Reservation_Details.LicenseNo = @LicNo;
```

```
Declare @fees int = 0;
```

```
Select @fees = dbo.Reservations_Charges.ParkingFee from  
dbo.Reservations_Charges where  
dbo.Reservations_Charges.ReservationId = @P_ReservationId;
```

```
if( @fees is not null )
```

```
Begin
```

```
return @fees
```

end

return 0

End;

---

## 14. Function: getFine

**Description:** This function retrieves the fine for a given vehicle based on its license number.

```
go
```

```
Create function getFine(@LicNo varchar(20) )
```

```
returns int
```

```
AS -- function that returns the fine scalar
```

```
Begin
```

```
Declare @P_ReservationId int = 0;
```

```
Select @P_ReservationId = dbo.Reservation_Details.ReservationId  
from dbo.Reservation_Details where  
dbo.Reservation_Details.LicenseNo = @LicNo;
```

```
Declare @fine decimal(10,2) = 0;
```

```
Select @fine = dbo.Reservations_Charges.Fine from  
dbo.Reservations_Charges where  
dbo.Reservations_Charges.ReservationId = @P_ReservationId;
```

```
if( @fine is not null )
```

```
Begin
```

```
        return @fine
```

```
    end
```

```
return 0
```

```
End;
```

---

## 15. View: allReservationListed

- **Description:** This view combines reservation details with time interval records and charges, listing all reservations with detailed information.

go

Create view allReservationListed

AS

```
Select dbo.Reservation_Details.ReservationId ,  
dbo.Reservation_Details.LicenseNo , dbo.Reservation_Details.SlotNumber  
, dbo.Reservation_Details.UserCNIC ,  
dbo.Reservation_TimeInterval_Record.ArrivalTime ,
```

```
dbo.Reservation_TimeInterval_Record.DepartureTime ,  
dbo.Reservations_Charges.ParkingFee , dbo.Reservations_Charges.Fine ,  
dbo.Reservation_Details.PaymentStatus
```

```
from dbo.Reservation_Details join dbo.Reservation_TimeInterval_Record  
On dbo.Reservation_Details.ReservationId =  
dbo.Reservation_TimeInterval_Record.ReservationId
```

```
join dbo.Reservations_Charges On dbo.Reservation_Details.ReservationId  
= dbo.Reservations_Charges.ReservationId
```

go

---

## 16. Stored Procedure: totalExpense

- **Description:** This procedure calculates the total amount (parking fee + fine) for a vehicle based on its license number.

go

Create procedure totalExpense

    @LicenseNo varchar(20) ,

    @TotalAmount int output

AS

begin -- procedure that calculate total amount using fee and fine  
function

    Select @TotalAmount =  
dbo.getFees(@LicenseNo)+dbo.getFine(@LicenseNo) from  
dbo.Reservation\_Details

    where dbo.Reservation\_Details.LicenseNo = @LicenseNo;

end;

Go

---

## 17. Stored Procedure: updatepaymentstatus

- **Description:** This procedure updates the payment status of a reservation to 'Paid' based on the license number.

go

```
create procedure updatepaymentstatus  -- update status to paid
```

```
    @LicenseNo varchar(20)
```

```
As
```

```
Begin
```

```
Update dbo.Reservation_Details set  
dbo.Reservation_Details.PaymentStatus = 'Paid' where  
dbo.Reservation_Details.LicenseNo = @LicenseNo;
```

```
End;
```

```
Go
```

---

## 18. Trigger: trg\_reservation\_cleanup

- **Description:** This trigger is activated instead of the `DELETE` operation on `Reservation_Details`. It performs several operations:
  - Inserts vehicle history into `Vehicle_Details`.
  - Inserts user history into `User_History`.
  - Inserts time history into `History_Records`.
  - Inserts charges history into `Charges_History`.
  - Updates the status of the parking slot to 'Available'.
  - Deletes corresponding records from `Reservations_Charges`, `Reservation_TimeInterval_Record`, `Reservation_Details`, `Vehicles`, and `Users`.

```
Create trigger trg_reservation_cleanup ON dbo.Reservation_Details
Instead of delete
AS
BEGIN

Insert into dbo.Vehicle_Details(          -- storing vehicle history
    ReservationId, LicenseNo, SlotNumber
)

Select
    deleted.ReservationId,
    deleted.LicenseNo,
```



```
        deleted.SlotNumber
from deleted

Insert into dbo.User_History (      --storing user history
        HistoryId,UserCNIC
)
Select
        dbo.Vehicle_Details.HistoryId,
        deleted.UserCNIC
from deleted
left join dbo.Vehicle_Details On dbo.Vehicle_Details.ReservationId =
deleted.ReservationId;

Insert into dbo.History_Records (      - storing time history
        HistoryId,ArrivalTime,DepartureTime
)
Select
        dbo.Vehicle_Details.HistoryId,
        dbo.Reservation_TimeInterval_Record.ArrivalTime,
        dbo.Reservation_TimeInterval_Record.DepartureTime
from deleted
```

```
left join dbo.Reservation_TimeInterval_Record On  
dbo.Reservation_TimeInterval_Record.ReservationId =  
deleted.ReservationId
```

```
left join dbo.Vehicle_Details On deleted.ReservationId =  
dbo.Vehicle_Details.ReservationId;
```

```
Insert into dbo.Charges_History( -- storing charges history  
    HistoryId,ParkingFee,Fine  
)
```

```
Select
```

```
dbo.Vehicle_Details.HistoryId,  
  
dbo.Reservations_Charges.ParkingFee,  
  
dbo.Reservations_Charges.Fine  
  
from deleted
```

```
left join dbo.Reservations_Charges On deleted.ReservationId =  
dbo.Reservations_Charges.ReservationId
```

```
left join dbo.Vehicle_Details On dbo.Vehicle_Details.ReservationId =  
deleted.ReservationId;
```

```
Update dbo.ParkingSlots set dbo.ParkingSlots.Status = 'Available'  
--updating the status to available after the slot is free
```

```
from dbo.ParkingSlots join deleted
```

```
ON dbo.ParkingSlots.SlotNumber = deleted.SlotNumber;
```

```
Delete dbo.Reservations_Charges          -- delete charges from current
reservations
```

```
from dbo.Reservations_Charges
```

```
join deleted ON dbo.Reservations_Charges .ReservationId =
deleted.ReservationId;
```

```
Delete dbo.Reservation_TimeInterval_Record    --delete time interval
from current reservations
```

```
from dbo.Reservation_TimeInterval_Record
```

```
join deleted ON deleted.ReservationId =
dbo.Reservation_TimeInterval_Record.ReservationId;
```

```
Delete dbo.Reservation_Details  -- delete details from current
reservations
```

```
from dbo.Reservation_Details
```

```
join deleted ON dbo.Reservation_Details.ReservationId =
deleted.ReservationId;
```

```
Delete dbo.Vehicles    -- delete that vehicle details from current
vehicles
```

```
from dbo.Vehicles
```

```
join deleted ON dbo.Vehicles .LicenseNo = deleted.LicenseNo
```

```
where not exists
```

```
(
```

```
    Select 1 from dbo.Reservation_Details where
dbo.Reservation_Details.LicenseNo = dbo.Vehicles.LicenseNo
```

```
);
```

```
Delete dbo.Users      -- delete user details from current users
```

```
from dbo.Users
```

```
join deleted ON dbo.Users.UserCNIC = deleted.UserCNIC
```

```
where not exists
```

```
(
```

```
    Select 1 from dbo.Reservation_Details where  
    dbo.Reservation_Details.UserCNIC = dbo.Users.UserCNIC
```

```
);
```

```
END;
```

---